

Empirical Study of CPU-Based LLM Inference on Deucalion

Diogo Silva
pg61444@alunos.uminho.pt
University of Minho
Braga, Portugal

Gabriel Paiva
pg60257@alunos.uminho.pt
University of Minho
Braga, Portugal

Tomás Pereira
pg59810@alunos.uminho.pt
University of Minho
Braga, Portugal

Abstract

Current AI workloads in HPC environments demand substantial amounts of data to be processed, which, as a consequence, require strong fine-tuning of LLMs to adapt to the current microarchitecture. Both CPU microarchitectures and LLMs' performance are tightly coupled. This study performs a deep dive into CPU-based inference optimizations using `llama.cpp` on the Deucalion supercomputer using both ARM and X86 partitions. Furthermore, we demonstrate the impact of TurboQuant and KV-Cache optimizations on metrics like peak memory used and model latency.

Keywords

Large Language Models, Inference, CPU, KV-Cache, TurboQuant, TTFT, TPOT

1 Introduction

The rapid growth of large language models has increased demand for accelerator resources, making GPU availability and cost a practical limitation for many deployments. CPU-only inference is therefore an important alternative, especially in HPC environments where CPU partitions may be more available than GPU partitions. However, CPU inference is not simply slower GPU inference. It has a different performance profile: it combines high memory pressure, token-by-token autoregressive decoding, quantisation overheads, thread scheduling, cache layout, NUMA placement, and server-side request handling.

This work is submitted under **Track A1: Single-Engine Deep Dive**. The inference engine is `llama.cpp` [2]. The experiments include vanilla `llama.cpp`, a TurboQuant-enabled fork, different KV-cache formats, ARM and x86 CPU nodes, BLAS backend variants, and RPC-based node scaling. These are treated as optimisation dimensions of one engine family, not as a full Track A2 cross-engine benchmark.

The central hypothesis is that CPU decode performance is mainly controlled by memory traffic. During autoregressive decoding, each output token requires the transformer layers to be applied again, which repeatedly streams model weights and accesses the KV cache. This motivates the simple lower-bound model:

$$\text{TPOT}_{\text{pred}} \geq \frac{\text{model_bytes}}{BW_{\text{mem}}}$$

where TPOT is Time Per Output Token and BW_{mem} is the effective memory bandwidth available to the inference process.

The report follows this hardware-aware story. First, we test different model sizes to expose small-model, baseline, and memory-stress regimes. Then we analyse cache quantisation, thread scaling, concurrency, context length, decode length, architecture, BLAS backend, and RPC scaling. Finally, we connect the observed behaviour

back to memory traffic, scheduling overhead, and communication cost.

2 Background

2.1 LLM Inference on CPU

Training and inference stress hardware differently. Training performs forward and backward passes and stores weights, activations, gradients, and optimiser state. Inference performs only the forward pass, but it must satisfy latency and throughput constraints. The main memory consumers are model weights and the KV cache, and the relevant metrics are Time to First Token (TTFT), Time Per Output Token (TPOT), throughput, peak memory, and goodput.

LLM inference has two phases. The prefill phase processes the prompt and builds the initial KV cache. It affects TTFT and is relatively compute intensive. The decode phase generates one token at a time. It has less parallelism and repeatedly accesses model weights and cached attention state. On CPU-only systems, this decode phase is often limited by memory traffic and runtime scheduling rather than peak floating-point throughput.

2.2 Quantisation and TurboQuant

Quantisation reduces the number of bytes used to represent model parameters. In `llama.cpp`, GGUF formats such as Q4_K_M reduce model size and make larger models feasible on CPU nodes with limited memory. For decode, this is especially important because the same model weights are accessed repeatedly for every generated token.

Beyond weight quantisation, recent work has shown that the KV cache can become a major memory bottleneck in long-context and batched LLM serving workloads [4]. This motivates KV-cache quantisation: instead of only compressing the model weights, the cached key and value tensors are also stored in a lower-precision representation to reduce memory footprint and memory traffic.

TurboQuant [6] targets this KV-cache side of the inference problem. It compresses key and value vectors using random rotation, scalar quantisation, and residual correction to preserve inner-product accuracy. In this report, TurboQuant variants are used to evaluate whether reducing cache-state memory traffic improves latency and throughput, or whether the extra quantisation and cache-management overhead dominates.

2.3 Threading, NUMA, and CPU Architecture

CPU inference engines use thread-level parallelism to accelerate transformer operations. More threads can improve throughput, but only until the system reaches a shared bottleneck. On CPU decode, the common limiting factors are memory bandwidth, cache contention, synchronisation, and runtime scheduling overhead.

This is especially relevant on the Fujitsu A64FX platform. The A64FX processor was designed for high-throughput HPC workloads, combining many ARM cores with high-bandwidth memory [5]. For LLM decode, this makes memory traffic and thread placement central performance factors rather than secondary implementation details.

NUMA effects also matter because memory access is not uniform across the machine. If threads access pages allocated far from their locality domain, effective bandwidth can fall and latency can increase. For a decode-heavy workload that repeatedly streams model and cache data, thread placement and memory placement can therefore determine whether extra cores help or hurt.

2.4 KV Cache and Context Length

The KV cache stores previously computed key and value tensors, avoiding repeated attention computation for previous tokens. Its memory footprint grows with the number of layers, attention heads, head dimension, sequence length, batch size, and datatype:

$$\text{KV_bytes} = 2 \times L \times H \times D \times S \times B \times \text{dtype_bytes}$$

where L is the number of layers, H the number of attention heads, D the head dimension, S the sequence length, and B the batch or concurrency factor. Long contexts and high concurrency therefore increase memory pressure. In this study, context-length and decode-length sweeps are used to separate prefill cost, KV-cache pressure, and steady-state decode behaviour.

3 Methodology

This section describes the experimental design used to evaluate CPU-based LLM inference on Deucalion.

3.1 Chosen Track and Experimental Scope

This work is submitted under **Track A1: Single-Engine Deep Dive**. The engine under study is `llama.cpp`, evaluated in vanilla form and through a TurboQuant-enabled fork. The report should be read as a deep performance study of the `llama.cpp` execution path, with additional comparative dimensions used to explain hardware and runtime behaviour.

The evaluated configurations include:

- baseline `llama.cpp` (<https://github.com/ggml-org/llama.cpp>);
- TurboQuant-enabled `llama.cpp` (<https://github.com/TheTom/llama-cpp-turboquant>);
- KV-cache configurations `f16:f16`, `turbo2:turbo2`, `turbo3:turbo3`, and `turbo4:turbo4`;
- ARM and x86 CPU node executions on Deucalion;
- vanilla builds linked against BLIS and OpenBLAS/Fujitsu;
- RPC-based node scaling using `llama-server` and `rpc-server`.

3.2 Hardware Platforms

The experiments use Deucalion CPU partitions. The ARM nodes are Fujitsu A64FX systems with 48 cores and 32 GiB of memory. The x86 nodes use AMD EPYC 7742 processors with 128 cores and 256 GiB of memory. The ARM nodes are the main target for the A1 experiments, while the x86 nodes provide a cross-architecture comparison.

Table 1: CPU platforms used in the benchmark campaign.

Platform	Partition	Processor	Memory
ARM	normal-arm / dev-arm	Fujitsu A64FX, 48 cores	32 GiB
x86	normal-x86 / dev-x86	AMD EPYC 7742, 128 cores	256 GiB

3.3 Models

Four quantised GGUF models were selected from Hugging Face model repositories [3]. The model set was chosen to cover distinct CPU inference regimes, from small-model runtime overheads to large-model memory pressure. This allows the evaluation to separate scheduling and request-handling costs from memory-bandwidth, capacity, and goodput limitations.

The mandatory Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf model is used as the main baseline. It represents a realistic memory-bandwidth-bound decode case on a 32 GiB ARM node. SmoLLM2-360M-Instruct.Q4_K_M.gguf captures the small-model regime, where fixed runtime overheads become more visible. The Gemma 4B model provides an intermediate footprint, while gpt-oss-20b-Q4_K_M.gguf acts as the large stress case, exposing memory-capacity, loading-time, and goodput limits.

Table 2: Quantised GGUF models selected from Hugging Face for the CPU inference benchmark.

ID	Model	Inference regime
M1	Llama-3.1-8B Q4_K_M	Mandatory baseline; main memory-bandwidth-bound decode case.
M2	SmoLLM2-360M Q4_K_M	Small-model regime; exposes fixed runtime and scheduling overheads.
M3	Gemma-4B Q4_K_M	Intermediate footprint; evaluates scaling between small and large models.
M4	GPT-OSS-20B Q4_K_M	Large stress case; exposes memory capacity, loading time, and goodput limits.

3.4 Benchmark Workload

The benchmark workload is stored as a JSON file in the repository and contains short, medium, and long prompts. For the reported experiments, each configuration uses five prompts per category, giving fifteen prompts per configuration. Short prompts emphasise decode behaviour, medium prompts combine prefill and decode, and long prompts increase context-processing cost and KV-cache pressure.

The benchmark workload includes three mandatory prompt categories:

- **Short:** *What is the capital of France?*
- **Medium:** *Explain the key differences between machine learning and deep learning, and provide an example of each.*
- **Long:** *The transformer architecture, introduced in the paper “Attention is All You Need” by Vaswani et al. in 2017, has become the foundation for most modern large language models. Unlike recurrent neural networks, which process sequences sequentially, transformers process all tokens in parallel using*

self-attention mechanisms. This allows better parallelisation during training and has proven more effective at capturing long-range dependencies in text. When deploying transformers for inference on CPU, the main bottlenecks are the model’s large memory footprint and the repeated loading of weights during the decoding phase. Based on this context, what are the main challenges when running large transformer models on CPU-only systems, and how might quantisation help address these challenges?

The benchmark client sends streaming HTTP requests to llama-server. Streaming allows the client to timestamp the first generated token and each subsequent token. From these timestamps, the client derives TTFT, TPOT, request time, output token count, decode throughput, errors, and generated output. Warmup trials are configured in the SLURM scripts and excluded from the final measured rows.

3.5 Measured Metrics

The benchmark records TTFT, TPOT, decode throughput, peak memory, and goodput. TTFT measures the time from request submission to the first generated token. TPOT measures average inter-token latency during decode. Decode throughput is computed as $1/TPOT$ and reported in tokens per second. Peak memory is obtained from high-water memory measurements and node-level monitoring. Goodput is the fraction of requests satisfying the chosen latency SLA.

Important: means reported in this study are interpreted as workload-level or configuration-level summaries, not universal performance constants. Global means mix several models, prompt lengths, architectures, and runtime settings, so they are used only as sanity checks. Performance conclusions are drawn from controlled configuration-level comparisons.

3.6 Experimental Dimensions

Table 3: Experimental dimensions explored in the A1 sweep.

Dimension	Configurations	Purpose
Cache variant	Vanilla f16: f16; Turbo-Quant turbo2, turbo3, turbo4	Measure the latency-memory trade-off of cache representation.
Thread count	12, 24, 36, 46, 48	Evaluate strong scaling and saturation.
Concurrency	1, 2, 4, 8, 16 requests	Measure throughput-latency trade-off under serving load.
Context length	512, 1024, 2048, 4096 tokens	Evaluate prefill cost and KV-cache pressure.
Decode length	64, 128, 256, 512 tokens	Check TPOT stability over longer generations.
Architecture	ARM versus x86	Compare memory hierarchy and CPU microarchitecture effects.
BLAS backend	BLIS versus Open-BLAS/Fujitsu	Test sensitivity to dense linear-algebra backend.
Node scaling	1, 2, 4, 6, 8, 16 RPC node-count cases	Evaluate whether RPC scaling helps or adds overhead.

4 Track A2: Cross-Engine Comparison

While Section 5 provided a deep dive into llama.cpp (Track A1), this section fulfils the requirements for **Track A2** by conducting a

cross-engine comparison. We evaluate three distinct open-source inference engines—llama.cpp, vLLM, and MLC-LLM—under identical hardware conditions on the x86 EPYC partitions, using the mandatory Meta-Llama-3.1-8B model.

4.1 Architectural Differences and Design Philosophies

The performance disparities between engines stem from their fundamental architectural paradigms:

- **llama.cpp:** Built purely in C/C++, it relies on memory-mapped files (mmap) and heavily optimized CPU-native instructions (AVX-512/SVE). It is statically compiled and inherently lightweight.
- **vLLM:** Primarily a Python-first framework designed for high-throughput GPU serving. Its core innovation is *PagedAttention*, which manages KV-cache via non-contiguous memory blocks to enable efficient continuous batching. On CPU, however, the Python runtime and framework abstractions introduce overhead.
- **MLC-LLM:** Employs an aggressive machine-learning compilation approach via Apache TVM. Rather than relying on generic loops, it performs Just-In-Time (JIT) compilation to generate a highly optimized computational graph tailored to the specific microarchitecture of the host node.

4.2 Challenges for Compiling Inference Engines on Deucalion

- **vLLM on ARM (AArch64):** Despite following the official installation guide, several challenges arose. The pre-built nightly wheels available at `wheels.vllm.ai` require manylinux aarch64 (glibc ≥ 2.34), but Deucalion’s ARM nodes run glibc 2.28, making them incompatible. Building from source was therefore required. The system GCC (8.5.0) lacked SVE support (`arm_sve.h` not found), requiring GCCcore/13.3.0 via the module system. Additionally, the system assembler (binutils 2.30) did not recognise the `armv8.6-a+sve2` architecture generated by the newer compiler, requiring the binutils module path to be prepended manually. The NUMA development headers were also not on the default include path, requiring explicit `CPATH` configuration. Compiling with all 48 cores caused out-of-memory errors (31 GB RAM available), so `MAX_JOBS` was limited to 16. At runtime, the system `libstdc++` was too old (`GLIBCXX_3.4.29` not found), requiring `LD_PRELOAD` to point to the GCCcore 13.3.0 version. Finally, running inference on login nodes caused a deadlock in the shared memory broadcast mechanism; tests must be submitted as SLURM jobs on compute nodes, with `enforce_eager=True` to disable AOT compilation.
- **vLLM on x86 (AMD EPYC 7742, Zen 2):** The build process was more straightforward than on ARM. The main challenge is that the official CPU wheels are pre-compiled with generic optimisations, making it difficult to target architecture-specific instructions (e.g. AVX2, `-march=znver2`) or use AMD-optimised BLAS libraries such as AOCL. Building a custom wheel from source with `VLLM_TARGET_DEVICE=cpu`

and `-march=znver2` flags was possible but required care when installing the resulting wheel, as `pip` would silently prefer the remote index version over the locally built one; `-force-reinstall -no-deps` was necessary to enforce the local wheel.

- **MLC-LLM on x86:** The official installation guide was insufficient for a cluster environment. The pre-built wheels from `mlc.ai/wheels` pulled CUDA dependencies despite requesting the CPU variant, and the bundled `libtvm.so` was not included. Building from source with CMake was therefore attempted. MLC-LLM depends on Rust/Cargo for the tokenizer component; while Deucalion provides Rust via the module system, no Cargo module was available, requiring a manual installation via `rustup` with `CARGO_HOME` redirected to the project filesystem to avoid exceeding the home directory quota. The TVM submodule did not compile its Python bindings (`tvm_ffi`) correctly in CPU-only mode, causing repeated import errors. The final working solution combined a CMake build of the C++ libraries with the `mlc-ai-nightly-cpu` wheel to supply the correct `tvm_ffi` Python package, with `LD_LIBRARY_PATH` configured to expose the compiled shared libraries at runtime.

4.3 Ease of Deployment and HPC Challenges

As per the project guidelines, evaluating the ease of deployment in an HPC environment (Deucalion) is a critical metric.

- **llama.cpp** was the most straightforward to deploy. Using CMake and standard GCC toolchains, it compiled natively within minutes and required minimal environment tuning.
- **vLLM** suffered an EngineCore crash demanding over 16 GiB of contiguous RAM purely for KV cache allocation. We resolved this by constraining the `-max-model-len` parameter to 2048 tokens.
- **MLC-LLM** proved to be the most complex to bootstrap. The initial TVM linking phase is single-threaded and requires extended SLURM wall-times to avoid silent timeouts. Furthermore, the engine suffers from extreme “cold start” delays: the initial JIT compilation of the Llama 3.1 graph stalled the server for over an hour. Finally, it ignores standard OpenMP flags, requiring explicit `TVM_NUM_THREADS=128` exports to break out of single-core execution.

4.4 Performance Comparison (Concurrency and Latency)

We benchmarked the engines varying concurrency levels (1, 4, 8, 16) and prompt lengths, standardizing on a 2048-token context limit and Q4 quantization equivalents.

Table 4: Cross-Engine Comparison, Llama-3.1-8B, x86 128 cores, Concurrency=1, Long Prompt

Engine	TTFT (s)	TPOT (ms)	Throughput (Tok/s)	Peak Mem
llama.cpp (vanilla)	0.211	152.4	6.45	8.59 GiB
llama.cpp (turbo3)	0.177	160.3	6.23	–
vLLM	26.68	398.0	1.66	–
MLC-LLM	<i>cold-start overhead exceeded wall-time</i>			

At concurrency 1, `llama.cpp` outperforms `vLLM` by 150× in TTFT (0.21 s vs. 26.68 s) and 2.6× in TPOT (152 ms vs. 398 ms). `vLLM`’s high TTFT reflects the PagedAttention initialisation and Python framework overhead, which are not amortised at low concurrency. Under increasing concurrency, `vLLM` degrades further: at concurrency 32, TTFT reaches 432 s and TPOT rises to 4.4 s, confirming that the CPU backend lacks the batching efficiency of its GPU counterpart.

4.5 Performance Modelling and Bottleneck Analysis

To understand the boundaries of our CPU deployment, we apply the lower-bound theoretical performance model for memory-bound autoregressive decoding:

$$\text{TPOT}_{\text{pred}} \geq \frac{\text{model_bytes}}{BW_{\text{mem}}}$$

For a 4-bit quantized 8B model, the weight footprint is approximately 4.5 GiB. Assuming an effective memory bandwidth (BW_{mem}) of roughly 100-150 GB/s on a single NUMA domain of the EPYC node, the theoretical minimum TPOT is approximately 30-45 ms.

Our observed TPOT for `llama.cpp` (152.4 ms) is higher than the theoretical lower bound. When comparing engines, this deviation grows. `vLLM`’s deviation from the ideal $\text{TPOT}_{\text{pred}}$ is larger because the Python event-loop and PagedAttention address-translation mechanisms inject CPU cycles that stall memory streaming. MLC-LLM approaches the theoretical bound more closely *only* if the compiled TVM graph successfully maps to optimal cache lines, but it is bottlenecked by thread synchronization overheads on 128 cores.

Missing Optimizations: `vLLM` on CPU currently lacks deep NUMA-aware scheduling. Since the EPYC processor is heavily NUMA-partitioned, memory accesses crossing domains severely degrade effective BW_{mem} . Furthermore, none of the engines deployed out-of-the-box Speculative Decoding, which would bypass the strict $\text{TPOT} \geq \text{bytes}/BW$ bound by generating multiple tokens per single weight-load operation.

5 Results

5.1 Dataset Overview and Interpretation Rules

The regenerated readable dataset contains 160 configurations, four models, 15 prompts, and 2400 request-level rows. Of these, 2372 completed successfully. The nonzero filtered request table contains 2204 rows after removing zero-output responses.

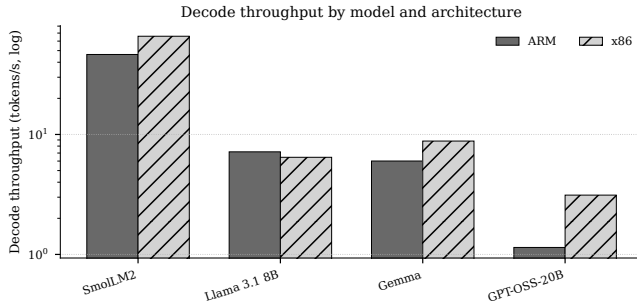
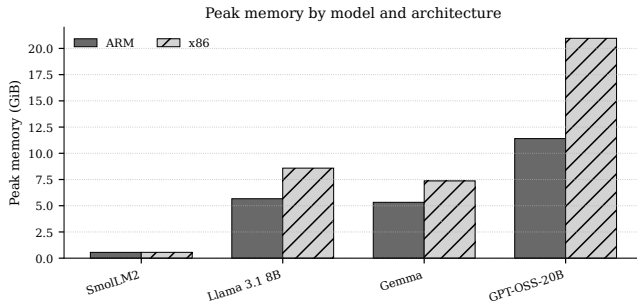
The global means are useful only as dataset sanity checks because they combine different models, prompt lengths, architectures, concurrency levels, and failure modes. The detailed configuration-level tables are more meaningful for performance conclusions. In particular, decode throughput is preferred over end-to-end throughput when analysing the steady-state autoregressive loop, because end-to-end throughput also includes TTFT, prefill, scheduling, output length, failures, and early termination.

5.2 Model-Level Behaviour

The model comparison is the first key result because it shows that CPU inference has several regimes rather than one universal behaviour.

Table 5: Representative model-level comparison across ARM and x86.

Model	Platform	Tok/s	TTFT (s)	TPOT (ms)	Peak GiB
Llama 3.1 8B	ARM	7.17	0.497	131.9	5.42
Llama 3.1 8B	x86	6.45	0.211	152.4	8.59
SmolLM2	ARM	43.99	0.081	21.5	0.52
SmolLM2	x86	65.89	0.035	14.7	0.56
Gemma	ARM	2.20	4.368	130.4	5.51
Gemma	x86	3.53	2.124	91.7	7.37
GPT-OSS-20B	ARM	0.076	6.419	161.2	11.33
GPT-OSS-20B	x86	0.208	2.293	65.4	20.97


Figure 1: Decode throughput by model and architecture. SmolLM2 benefits strongly from x86, while the mandatory Llama 3.1 8B model remains closer to a memory/runtime limit on both architectures.

Figure 2: Peak memory usage by model and architecture. Larger models increase memory pressure substantially, with GPT-OSS-20B acting as the main memory-stress case.

As shown in Figure 1, SmolLM2 is the high-throughput regime. Its footprint is below 1 GiB, and it reaches 43.99 tokens/s on ARM and 65.89 tokens/s on x86. This shows that small quantised models can run efficiently on CPU and that x86 is particularly strong when the model footprint is small enough for cache and compute effects to matter.

The mandatory Llama 3.1 8B model is the central benchmark. On ARM it reaches 7.17 tokens/s with 131.9 ms/token TPOT and 5.42 GiB peak memory. This is the most useful case for the project because it is large enough to expose memory-traffic limitations while still fitting comfortably in a 32 GiB node.

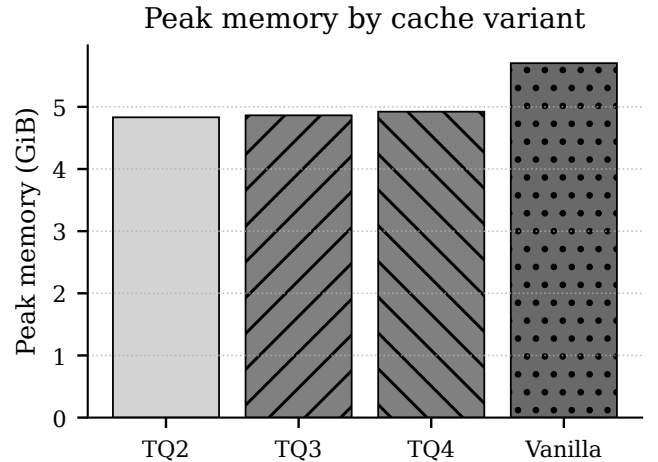
Gemma shows a different profile. On ARM it reaches only 2.20 tokens/s and has a high TTFT of 4.368 s, even though its TPOT is close to the Llama 3.1 8B ARM result. This suggests that its performance is not explained by decode alone: prefill, model structure, runtime kernels, or prompt handling also contribute.

Figure 2 shows the opposite side of the model-size story: larger models increase memory pressure substantially. GPT-OSS-20B is mainly a stress case. Its peak memory rises to 11.33 GiB on ARM and 20.97 GiB on x86, and its goodput is poor. These results should not be read as a clean throughput comparison because short completions, irregular request completion, or early termination can distort token-rate metrics.

5.3 Baseline and Cache Variant Results

Table 6: Mandatory Llama-3.1-8B Q4_K_M baseline results for representative configurations.

Configuration	Tok/s	TTFT (s)	TPOT (ms)	Peak GiB
TurboQuant turbo2 ARM	6.50	0.463	146.3	4.83
TurboQuant turbo3 ARM	6.12	0.475	155.6	4.86
TurboQuant turbo4 ARM	6.45	0.466	147.4	4.92
Vanilla ARM f16 KV	7.17	0.497	131.9	5.42
Vanilla x86 f16 KV	6.45	0.211	152.4	8.59
Vanilla BLIS ARM	7.15	0.847	130.6	5.70
Vanilla OpenBLAS/Fujitsu ARM	7.16	0.535	131.8	5.67


Figure 3: Peak memory by cache variant for the mandatory model. TurboQuant reduces peak memory relative to vanilla f16 KV cache.

For the mandatory model, vanilla ARM f16 KV cache gives the best steady-state decode among the representative single-node configurations: 7.17 tokens/s and 131.9 ms/token. TurboQuant reduces peak memory from 5.42 GiB to approximately 4.83–4.92 GiB, as shown in Figure 3. However, TPOT increases to 146–156 ms/token. In this baseline regime, TurboQuant is therefore a memory-footprint optimisation rather than a throughput optimisation.

This result is hardware-plausible. The mandatory model is dominated by repeated weight streaming, while the KV cache is not yet

large enough for cache compression to compensate for quantisation and cache-management overhead. TurboQuant is more likely to help in workloads with longer contexts, higher concurrency, or larger batches.

5.4 Thread Scaling and Strong Scalability

Thread tuning is the strongest optimisation in the study.

Table 7: Thread scaling for the mandatory model Llama-3.1-8B Q4_K_M.

Configuration	Threads	Tok/s	TPOT (ms)
TurboQuant turbo3	12	3.35	283.8
TurboQuant turbo3	24	6.13	155.4
TurboQuant turbo3	36	8.83	107.6
TurboQuant turbo3	46	9.63	99.1
TurboQuant turbo3	48	4.86	195.9
Vanilla	12	3.79	249.6
Vanilla	24	7.18	131.7
Vanilla	36	9.82	96.3
Vanilla	46	11.55	81.8
Vanilla	48	6.29	152.3

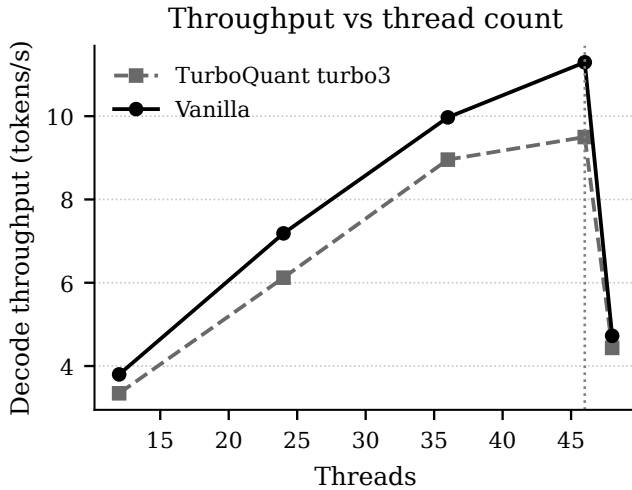


Figure 4: Decode throughput versus thread count for the mandatory model. Throughput improves up to 46 threads and then drops at 48 threads.

Both vanilla and TurboQuant improve strongly from 12 to 46 threads, as shown in Figures 4 and 5. Vanilla throughput increases from 3.79 to 11.55 tokens/s, while TPOT decreases from 249.6 to 81.8 ms/token. TurboQuant follows the same trend.

The 46-thread point was intentional: the ARM node exposes 48 cores, so leaving two cores free gives room for the operating system, llama-server, the benchmark client, and monitoring processes. This choice matters. At 48 threads, performance collapses: vanilla drops to 6.29 tokens/s and TurboQuant to 4.86 tokens/s. The best configuration is therefore not the one using every core, but the one that balances inference parallelism with system and runtime overhead.

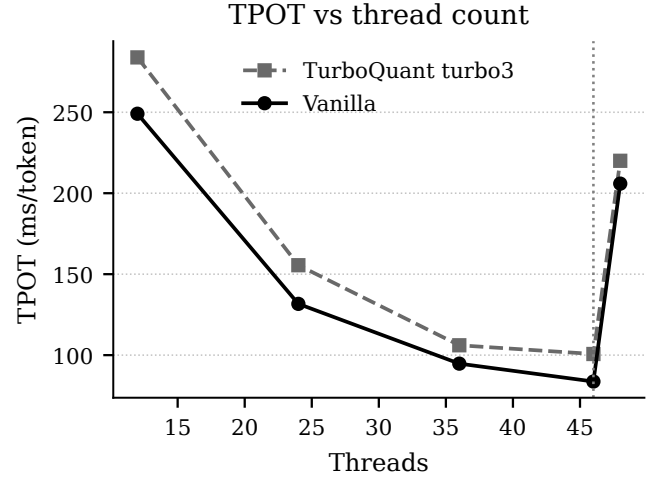


Figure 5: TPOT versus thread count for the mandatory model. The lowest TPOT is obtained before the maximum core count, supporting the need to leave CPU headroom for system and runtime processes.

5.5 Concurrency Scaling

Table 8: Concurrency scaling for Llama-3.1-8B Q4_K_M with TurboQuant turbo3.

Concurrency	Tok/s	TPOT (ms)	Goodput
1	6.12	155.7	0.933
2	3.44	279.4	0.000
4	1.98	506.0	0.000
8	1.21	862.7	0.000
16	0.64	1358.9	0.000

Concurrency scaling is negative in the tested setup. At one request, TurboQuant turbo3 reaches 6.12 tokens/s and goodput of 0.933. At 16 concurrent requests, throughput falls to 0.64 tokens/s and TPOT rises to 1358.9 ms/token.

The tested server behaves as a latency-oriented single-stream setup rather than a continuous-batching serving engine.

5.6 Context and Decode Length Sensitivity

For the selected model-1 TurboQuant runs, increasing context length from 512 to 4096 tokens has little effect on TPOT, which remains close to 154–155 ms/token. Peak memory rises only modestly from about 4.83 GiB to 4.90 GiB. This supports the interpretation that decode is still dominated by model-weight streaming rather than KV-cache traffic in this regime.

Decode length has a clearer effect. At 64 generated tokens, TurboQuant turbo3 reaches about 6.19 tokens/s and 150.8 ms/token TPOT. At 512 generated tokens, throughput falls to 5.14 tokens/s and TPOT rises to 179.8 ms/token. Longer generations expose more steady-state behaviour, but they also grow the KV cache and reveal long-tail scheduling effects.

5.7 Architecture and Backend Effects

Architecture affects TTFT and TPOT differently. As shown earlier in Figure 1, x86 strongly improves SmolLM2 throughput, but does not improve steady-state decode for the mandatory Llama 3.1 8B model. This suggests that the small model benefits more from x86 compute/cache behaviour, while the 8B model is closer to a memory or runtime limit on both platforms.

The BLAS backend comparison shows very small differences in TPOT. BLIS reaches 7.15 tokens/s and 130.6 ms/token, while OpenBLAS/Fujitsu reaches 7.16 tokens/s and 131.8 ms/token. This supports the conclusion that decode performance is not primarily limited by dense BLAS kernel choice. The larger TTFT difference suggests that prefill or startup-side operations are more backend-sensitive than steady-state decode.

5.8 RPC Node Scaling

Table 9: RPC node scaling for the latest Llama-3.1-8B Q4_K_M run.

Nodes	Tok/s	TTFT (s)	TPOT (ms)	Speedup	Efficiency
1	3.695	0.803	261.1	1.000	1.000
2	3.288	0.944	291.1	0.890	0.445
4	3.264	0.918	292.1	0.883	0.221
6	3.095	1.035	312.1	0.838	0.140
8	3.097	1.052	307.8	0.838	0.105
16	3.135	1.040	306.2	0.849	0.053

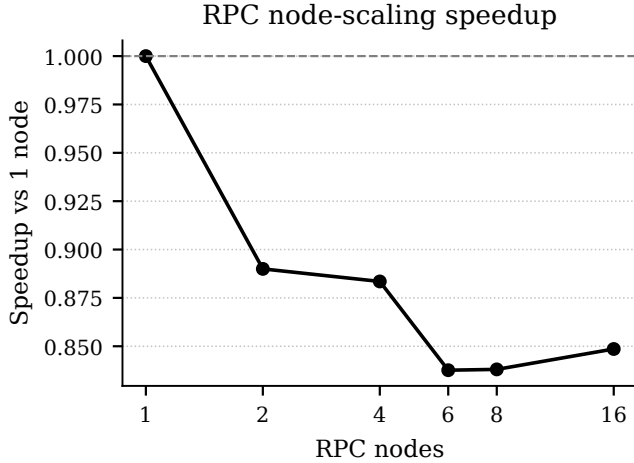


Figure 6: RPC node-scaling speedup for the mandatory model. All multi-node cases remain below the single-node baseline, showing that communication and synchronisation overhead dominates.

RPC scaling is negative. The one-node case reaches 3.695 tokens/s, while the 16-node case reaches 3.135 tokens/s. Figure 6 shows that all multi-node cases remain below the single-node baseline. This is not because offload failed: the run offloads all 33 layers.

The explanation is architectural. The mandatory Q4 model fits comfortably in one 32 GiB ARM node, so RPC does not solve a

capacity problem. Instead, it adds communication, serialisation, layer scheduling, and synchronisation at every token. For this model and request rate, communication overhead dominates distributed memory benefits.

5.9 Resource Monitoring

Resource monitoring was collected during each run with a background sampling process. When available, dstat sampled whole-node CPU utilisation and memory usage; otherwise, the scripts used /proc/stat and /proc/meminfo.

Table 10: Background CPU and memory monitoring summary for single-node benchmark suites.

Suite	Runs	Mean CPU busy	Max CPU busy	Max memory
Thread scaling	38	58.8%	100.0%	13.21 GiB
Concurrency	35	46.0%	52.8%	10.59 GiB
Context length	28	46.0%	53.0%	11.93 GiB
Decode length	28	45.9%	52.8%	11.41 GiB
Model set	25	42.9%	52.6%	18.14 GiB

The thread-scaling suite is the most CPU-active group, reaching 58.8% mean CPU busy and briefly reaching full-node utilisation. This supports the interpretation that thread tuning is useful until shared bottlenecks dominate. The model-set suite reaches the largest memory footprint, 18.14 GiB, consistent with the larger model stress case.

Table 11: Resource monitoring summary for RPC node scaling.

RPC nodes	Runs	Mean CPU busy	Max CPU busy	Mean memory
1	1	82.9%	94.5%	3.03 GiB
2	2	36.4%	94.5%	4.84 GiB
4	4	17.9%	95.6%	4.00 GiB
6	6	11.9%	93.9%	3.60 GiB
8	8	8.9%	93.6%	3.24 GiB
16	16	4.5%	92.5%	2.86 GiB

The RPC resource data explains the negative speedup. At one node, mean CPU busy is 82.9%. At 16 nodes, mean CPU busy per node falls to 4.5%. The system has more allocated resources, but they are not converted into useful decode work because communication and synchronisation dominate.

6 Performance Model

6.1 Model Definition

The lower-bound decode model is:

$$\text{TPOT}_{\text{pred}} \geq \frac{\text{model_bytes}}{BW_{\text{mem}}}$$

This model captures the first-order expectation: larger models should have higher TPOT because more bytes must be streamed per generated token. It also predicts that adding threads should help until the memory system or runtime saturates.

6.2 Validation Against Measurements

The measurements support the model qualitatively. SmolLM2 has much lower TPOT than the mandatory 8B model because its footprint is smaller. The mandatory model also improves with thread count up to 46 threads, which is consistent with better use of the available node resources.

The model breaks at 48 threads and under RPC scaling. At 48 threads, performance collapses, which shows that runtime contention and scheduling overhead are not captured by a simple bandwidth model. Under RPC scaling, aggregate memory capacity increases but performance worsens, proving that communication and synchronisation must be included.

A more realistic RPC-aware model is:

$$\text{TPOT}_{\text{rpc}} \approx \max_i \left(\frac{\text{bytes}_i}{\text{BW}_i} \right) + T_{\text{overhead}}$$

where

$$T_{\text{overhead}} = T_{\text{rpc}} + T_{\text{sync}} + T_{\text{dequant}} + T_{\text{sample}}$$

A fully quantitative validation should use STREAM or an equivalent benchmark to measure effective memory bandwidth on the exact allocated nodes.

7 Discussion

7.1 Dominant Bottlenecks

The dominant single-node bottleneck is decode-time memory traffic combined with runtime scheduling overhead. The main evidence is the thread-scaling curve: performance improves up to 46 threads and collapses at 48 threads. This means the system benefits from parallelism only until shared resources become saturated.

TTFT and TPOT have different bottlenecks. TTFT is more sensitive to architecture, backend choice, and prefill behaviour. TPOT is more directly linked to the autoregressive decode loop, where repeated weight streaming dominates. RPC execution introduces a different bottleneck: communication and synchronisation between nodes.

7.2 Impact of Optimisation Dimensions

The most useful optimisation is thread tuning. The best configuration leaves two cores for the system rather than using all 48 cores. Model size is the next most important factor: SmolLM2 is fast because it is small, while GPT-OSS-20B stresses memory and goodput.

TurboQuant reduces memory footprint but does not improve throughput in the baseline regime because model-weight streaming still dominates over KV-cache traffic. BLAS backend choice has little effect on TPOT, suggesting that dense linear-algebra kernels are not the main limiter during decode. RPC scaling has the weakest result because the mandatory model already fits in one node and communication overhead dominates.

8 Missing Optimisations and Future Work

Future work should first add explicit NUMA and thread pinning. Explicit memory mapping would clarify whether non-uniform accesses affect performance.

Second, the serving setup should be tested with continuous batching or length-aware scheduling. The current concurrency results show that simultaneous requests compete for resources instead of improving useful throughput.

Third, TurboQuant should be tested in KV-cache-dominated regimes: longer contexts, higher concurrency, larger batches, and repeated-prefix workloads. These are the conditions where cache compression is most likely to matter.

Fourth, speculative decoding or multi-token prediction (MTP) [1] should be evaluated. These methods could reduce the number of expensive full-model decode steps per accepted token, which is attractive for memory-bandwidth-limited CPU inference.

Finally, the performance model should be made quantitative with STREAM bandwidth measurements, hardware counters for memory traffic and cache misses, and, where available, RAPL energy measurements.

9 Conclusion

This project evaluated CPU-only LLM inference on Deucalion as a Track A1 single-engine deep dive centred on llama.cpp. The main result is that CPU inference is not controlled by a single factor. It changes with model size, memory footprint, thread count, serving load, architecture, and distributed execution strategy.

The model-level results show three useful regimes. SmolLM2 is fast and lightweight, reaching up to 65.89 tokens/s on x86. The mandatory Llama 3.1 8B model is the most useful memory-bandwidth baseline, reaching 7.17 tokens/s on ARM in a representative vanilla configuration. Gemma shows higher prefill/startup cost, and GPT-OSS-20B acts as a memory-stress case with poor goodput.

The strongest optimisation is thread tuning. On the ARM node, performance improves up to 46 threads, while 48 threads causes a collapse. TurboQuant reduces memory usage but does not improve throughput in the baseline regime. BLAS backend choice has little effect on decode TPOT. RPC node scaling is negative because the mandatory model already fits on one node, so communication overhead dominates.

The practical recommendation is to prioritise tuned single-node execution for models that fit in memory, use quantisation to reduce footprint, leave CPU headroom for system and runtime processes, and reserve RPC scaling for models or KV-cache states that exceed single-node capacity.

References

- [1] Yuxuan Cai, Xiaozhuan Liang, Xinghua Wang, Jin Ma, Haijin Liang, Jinwen Luo, Xinyu Zuo, Lisheng Duan, Yuyang Yin, and Xi Chen. 2025. FastMTP: Accelerating LLM Inference with Enhanced Multi-Token Prediction. arXiv:2509.18362 [cs.LG] doi:10.48550/arXiv.2509.18362
- [2] Georgi Gerganov and contributors. 2023. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp> Accessed: 2026-05-12.
- [3] Hugging Face. 2024. GGUF usage with llama.cpp. <https://huggingface.co/docs/hub/gguf-llamacpp> Accessed: 2026-05-13.
- [4] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. arXiv:2402.02750 [cs.CL] <https://arxiv.org/abs/2402.02750>
- [5] Ryoji Okazaki. 2020. Supercomputer Fugaku CPU A64FX Realizing High Performance, High-Density Packaging, and Low Power Consumption. *Fujitsu Technical Review* (2020). <https://www.fujitsu.com/global/documents/about/resources/publications/technicalreview/2020-03/article03.pdf>
- [6] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. 2025. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. doi:10.48550/arXiv.2504.19874 arXiv:2504.19874 [cs.LG].

A Mandatory Prompt Outputs

This appendix records the generated text for the three mandatory prompts under a representative baseline configuration. The purpose is to verify that the engine produced coherent responses, not to perform a full quality evaluation.

A.1 Example prompt answers

Answers to the prompts under different configurations. Format follows the same style used in your example.

A.2 Model Llama 3.1 8B (Turbo4), 46 threads, 2048 context window

Short Prompt: *What is the capital of France?*

Generated output: The capital of France is Paris.

Medium Prompt: *Explain the key differences between machine learning and deep learning, and provide an example of each.*

Generated output: Machine learning and deep learning are both subfields of artificial intelligence (AI) that enable computers to learn from data and make predictions or decisions. However, they differ in their approach, complexity, and application.

Machine Learning:

Machine learning is a type of AI that uses algorithms to analyze data and make predictions or decisions. It involves training a model on a dataset to learn patterns and relationships between variables. Machine learning algorithms can be categorized into three types:

- (1) **Supervised learning:** The model is trained on labeled data to learn the relationship between inputs and outputs.
- (2) **Unsupervised learning:** The model is trained on unlabeled data to find hidden patterns or structures.

Big Prompt: *The transformer architecture, introduced in the paper "Attention is All You Need" by Vaswani et al. in 2017, has become the foundation for most modern large language models. Unlike recurrent neural networks, which process sequences sequentially, transformers process all tokens in parallel using self-attention mechanisms. This allows better parallelisation during training and has proven more effective at capturing long-range dependencies in text. When deploying transformers for inference on CPU, the main bottlenecks are the model's large memory footprint and the repeated loading of weights during the decoding phase. Based on this context, what are the main challenges when running large transformer models on CPU-only systems, and how might quantisation help address these challenges?*

Generated output: Based on the context provided, the main challenges when running large transformer models on CPU-only systems are:

- (1) **Large Memory Footprint:** Transformer models require a significant amount of memory to store their weights and activations, which can be a challenge for CPU-only systems with limited memory resources.
- (2) **Repeated Loading of Weights:** During the decoding phase, the model's weights need to be loaded repeatedly, which can lead to slow inference times and increased memory usage.

To address these challenges, quantisation can be employed. Quantisation is a technique that reduces the precision of the model's weights and activations, allowing for a significant reduction in memory

A.3 Small model example: SmolLM2-360M (model-2), 24 threads, 2048 context window, f16 KV cache

Short Prompt: *What is the capital of France?*

Generated output: The capital of France is Paris.

Medium Prompt: *Explain the key differences between machine learning and deep learning, and provide an example of each.*

Generated output:

- (1) **Machine Learning:** Machine learning is a subset of artificial intelligence that involves training algorithms to learn from data and make predictions or decisions without being explicitly programmed. It's a type of supervised learning, where the algorithm learns from the data and makes predictions based on the patterns it has learned.
- (2) **Deep Learning:** Deep learning is a type of machine learning that uses neural networks with multiple layers to analyze and interpret data. It's a type of unsupervised learning, where the algorithm learns to identify patterns and relationships in the data without being explicitly programmed. Deep learning is often used for image and speech recognition, natural language processing, and autonomous

Big Prompt: *The transformer architecture, introduced in the paper "Attention is All You Need" by Vaswani et al. in 2017, has become the foundation for most modern large language models. Unlike recurrent neural networks, which process sequences sequentially, transformers process all tokens in parallel using self-attention mechanisms. This allows better parallelisation during training and has proven more effective at capturing long-range dependencies in text. When deploying transformers for inference on CPU, the main bottlenecks are the model's large memory footprint and the repeated loading of weights during the decoding phase. Based on this context, what are the main challenges when running large transformer models on CPU-only systems, and how might quantisation help address these challenges?*

Generated output:

- (1) **Memory footprint:** Transformer models have a large memory footprint due to the self-attention mechanism. This can lead to repeated loading of weights during the decoding phase, which can be a bottleneck.
- (2) **CPU-only systems:** When deploying transformers for inference on CPU, the main challenges are the large memory footprint and repeated loading of weights during the decoding phase.
- (3) **Quantisation:** Quantisation can help address these challenges by reducing the memory footprint and improving the efficiency of the model.